Outlab for Lab 3

This outlab is ungraded but I would advice everyone to try and solve it fully since it would be very helpful in brushing up on your programming.

Very broadly we will be exploring methods for Multi Class Regression on a somewhat real world dataset. We begin with audio files containing 2 second samples of various instruments. Code to extract relevant audio features from the audio is already given. You can learn more about the features here.

For the lab you have to fill in the various parts of the multi class regression functions and try to build a classifier class. In each section relevant numpy functions and broad knowledge needed to solve the subpart is provided in the comments. If you are unfamiliar with these concepts / functions then read up on them!

When you run main.py it will print out the accuracies and number of correct and incorrectly classified samples. Roughly for correct implementations the number of incorrectly classified samples should be less than around 20. (The exact amount my vary depending on the specifics of your implementation).

For each function you implement also try to add a few sanity check cases in main.py similar to the ones I have added specifically for make_label. It will be very valuable and helpful for debugging. In general if a function works correctly for a small case then it should work correctly for larger cases too.
While we may add similar helping hands in the inlabs itself it is incredibly useful and valuable to learn how to manually add these checks and reason about functions individually. A lot of students had most of their functions correct but the final accuracy was low because of just a single function being incorrectly implemented.

Primer on Multi Class Regression

Till now we have dealt with situations where we need to split the data into two distinct classes. In this question we will explore how to extend this methodology to situations where classification needs to be done for more than two classes. Such scenarios are generally called Multi Class Regression.

We will study three techniques to achieve this. The first two are general and work to extend any binary classification algorithm. The third one is a specific extension of binary logistic regression. It is also often called softmax regression or multiple logistic regression.

Let us assume we have data

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix} \in \mathbb{R}^{n \times d}$$

and Class labels

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \qquad y_i \in \{1, 2, \ldots, k\}$$

Extending a General Binary Classifier

The below two techniques can extend any binary classifier into a multi class classifier.

One vs Many Classifiers

In One vs Many approach, we train k different binary classifiers. The jth binary classifier distinguishes instances of the jth class from instances of all other classes. More formally to train the jth model we relabel the data as follows:

$$y_i^{(j)} = \begin{cases} 1, & \text{if } y_i = j \\ 0, & \text{otherwise} \end{cases}$$

To classify a new instance each classifier produces a probability for a test example (Or more generally a score). Each score roughly indicates the confidence of our models that this new instance lies in this particular class. The prediction is simply the class with the highest score. So:

$$\hat{y} = \arg \max_{j \in \{1,\ldots,k\}} f_j(x)$$

where $f_j(x)$ denotes the output of the binary classifier trained for class j. Under this scheme we will have to train k classifiers if there are k classes.

One vs One Classifiers

In the One-vs-One approach we train separate binary classifiers for every pair of distinct classes. Each classifier is trained using only data from the two classes it is trying to distinguish.

Then upon seeing a new instance x. We make the prediction $\hat{y}$ as follows:

$$\hat{y} = \arg \max_{j \in \{1,\ldots,k\}} \sum_{i \neq j} \mathbb{I}\left(f_{ij}(x) = j\right)$$

where $f_{ij}(x)$ denotes the prediction of the classifier trained on classes i and j, and $\mathbb{I}(\cdot)$ is the indicator function.
You can think of this as each classifier "Voting" on what class it thinks x lies in.... And then finally the class with the most votes is chosen to be the prediction.

Multi Class Logistic Regression

Unlike the other two methods this method is a natural extension of logistic regression to multiple classes.
For k classes, we have a parameter vector $w_j \in \mathbb{R}^d$ for each class. Given an input x, the conditional probability of class j is modelled using the softmax function.

$$P(y = j \mid x) = \frac{\exp(\mathbf{w}_j^\top x)}{\sum_{\ell=1}^{k} \exp(\mathbf{w}_\ell^\top x)}$$

Now suppose we wish to maximize the log likelihood of the data.

The likelihood of the full dataset is given by

$$\mathcal{L}(\{\mathbf{w}_j\}) = \prod_{i=1}^{n} \prod_{j=1}^{k} P(y_i = j \mid x_i)^{\mathbb{I}(y_i=j)}$$

Taking the logarithm of the likelihood, we obtain the log-likelihood:

$$\log \mathcal{L} = \sum_{i=1}^{n} \sum_{j=1}^{k} \mathbb{I}(y_i = j) \log P(y_i = j \mid x_i)$$

Substituting the softmax expression, we get

$$\log \mathcal{L} = \sum_{i=1}^{n} \sum_{j=1}^{k} \mathbb{I}(y_i = j) \left(\mathbf{w}_j^\top x_i - \log \sum_{\ell=1}^{k} \exp(\mathbf{w}_\ell^\top x_i) \right)$$

Since optimization is typically performed by minimization, we define the loss function as the negative log-likelihood:

$$\mathcal{J} = -\sum_{i=1}^{n} \sum_{j=1}^{k} \mathbb{I}(y_i = j) \log P(y_i = j \mid x_i)$$

For leaning rate η we will have the following updates for each value of $j \in \{1, 2, \cdots k\}$

$$w_j' = w_j - \eta \nabla_{w_j} \mathcal{J}$$

Where the gradient is

$$\frac{\partial \mathcal{J}}{\partial \mathbf{w}_j} = \sum_{i=1}^{n} \left(P(y_i = j \mid x_i) - \mathbb{I}(y_i = j) \right) x_i$$

To efficiently vectorize the above operations, we represent the class labels using a one-hot encoding. Let $Y \in \mathbb{R}^{n \times k}$ denote the one-hot encoded label matrix, where

$$Y_{ij} = \begin{cases} 1, & \text{if } y_i = j \\ 0, & \text{otherwise} \end{cases}$$

Let $W \in \mathbb{R}^{d \times k}$ be the weight matrix whose j-th column is $\mathbf{w}_j$. The matrix of class scores is given by

$$Z = XW \in \mathbb{R}^{n \times k}$$

Applying the softmax function row-wise yields the predicted class probabilities

$$P_{ij} = \frac{\exp(Z_{ij})}{\sum_{\ell=1}^{k} \exp(Z_{i\ell})} \quad \text{for } i = 1, \ldots, n$$

The categorical cross-entropy loss can now be written compactly as

$$\mathcal{J} = -\sum_{i=1}^{n} \sum_{j=1}^{k} Y_{ij} \log P_{ij}$$

The gradient of the loss with respect to the weight matrix W is

$$\nabla_W \mathcal{J} = X^\top (P - Y)$$

Q3. Classification of Musical Instrument Data using Multi Class methods

For this question you are provided with a dataset containing 2 second audio samples of recordings of four different instruments. We will build a classification system that begins with just the audio files and classifies it based on which instrument is making the sound.

Feature Extraction

main.py has some code that extracts relavant features from audio files and makes it into a dataframe. You can have a look at that function but the specifics of how the features are actually calculated is not needed.

After that there are three classes and a few helper functions you need to fill in.

For my implementation one-vs-one and one-vs-many are well tuned while softmax is not. Your mileage may vary. When you are confident that your implementation is correct try and tune the learning rate and iterations. I was getting the following accuracy values and yours should be similar.

- One-vs-Rest Accuracy: 0.9635
- One-vs-One Accuracy: 0.9708
- Softmax Accuracy: 0.9635

What you Need to do

Fill out the template code to implement multi class logistic regression, one vs many classifiers and one vs one classifiers.

Train each of these on the sample data and analyze the accuracy achieved.